Lab Exercise 6 — Multilayer Perceptron (MLP) Using Artificial Neural Networks

Course: MAI511-2 Advanced Machine Learning
Problem type: Binary classification
Dataset: Wisconsin Diagnostic Breast Cancer (WDBC), provided by scikit-learn
Aim: Design, preprocess, train, evaluate, and analyse an MLP, including its architecture and trainable parameters.

1. Problem definition and dataset

Predict whether a breast mass is **malignant** or **benign** from numeric measurements computed from digitized images of fine needle aspirates.

- **Input features:** 30 numeric measurements (for example, mean radius, texture, perimeter, area, and concavity).
- **Target:** diagnosis (`0 = malignant`, `1 = benign`).
- **Task:** binary classification.

The dataset has no categorical predictors, and the selected scikit-learn version contains no missing values. The pipeline nevertheless includes median imputation so it remains robust if missing values occur in future input data.

```
import sys, warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import tensorflow as tf
import sklearn
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             confusion_matrix, classification_report, roc_auc_score,
                             roc_curve)
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Input, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
tf.random.set_seed(RANDOM_STATE)

print("Python:", sys.version.split()[0])
print("TensorFlow:", tf.__version__)
print("scikit-learn:", sklearn.__version__)
```

```
Python: 3.13.18
TensorFlow: 2.21.0
scikit-learn: 1.6.1
```

2. Load data and inspect it

```
dataset = load_breast_cancer(as_frame=True)
X = dataset.data.copy()
# sklearn target: 0=malignant, 1=benign
y = dataset.target.astype(int).copy()

df = X.copy()
df["diagnosis"] = y.map({0: "malignant", 1: "benign"})
display(df.head())
print("Dataset shape:", df.shape)
print("\\nTarget counts:")
display(df["diagnosis"].value_counts().rename_axis("diagnosis").to_frame("count"))
print("\\nMissing values:", int(X.isna().sum().sum()))
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst texture	worst perimeter	worst area	worst smoothness	worst compactness	worst concavity	worst concave points	worst symmetry	worst fractal dimension	diagnosis
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...	17.33	184.60	2019.0	0.1622	0.6656	0.7119	0.2654	0.4601	0.11890	malignant
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...	23.41	158.80	1956.0	0.1238	0.1866	0.2416	0.1860	0.2750	0.08902	malignant
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	...	25.53	152.50	1709.0	0.1444	0.4245	0.4504	0.2430	0.3613	0.08758	malignant
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597	0.09744	...	26.50	98.87	567.7	0.2098	0.8663	0.6869	0.2575	0.6638	0.17300	malignant
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883	...	16.67	152.20	1575.0	0.1374	0.2050	0.4000	0.1625	0.2364	0.07678	malignant

5 rows × 31 columns

```
Dataset shape: (569, 31)
\nTarget counts:
```

diagnosis	count
benign	357
malignant	212

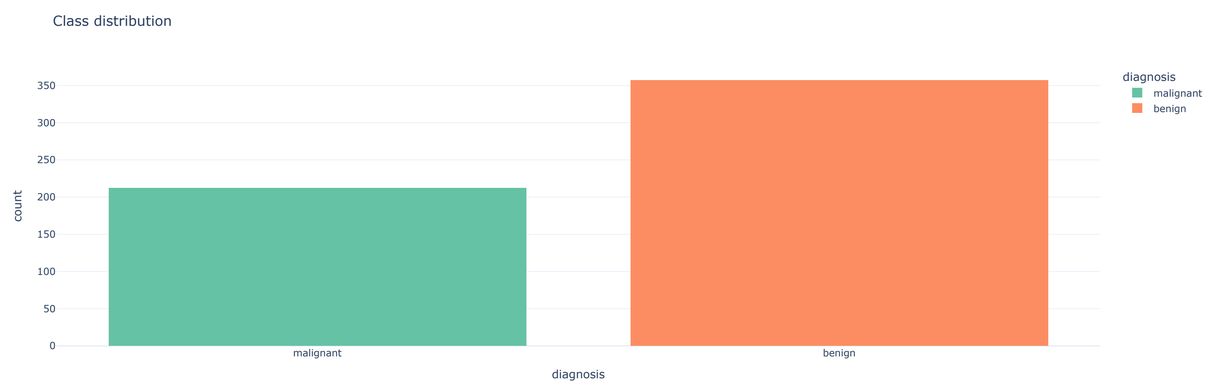
```
\nMissing values: 0
```

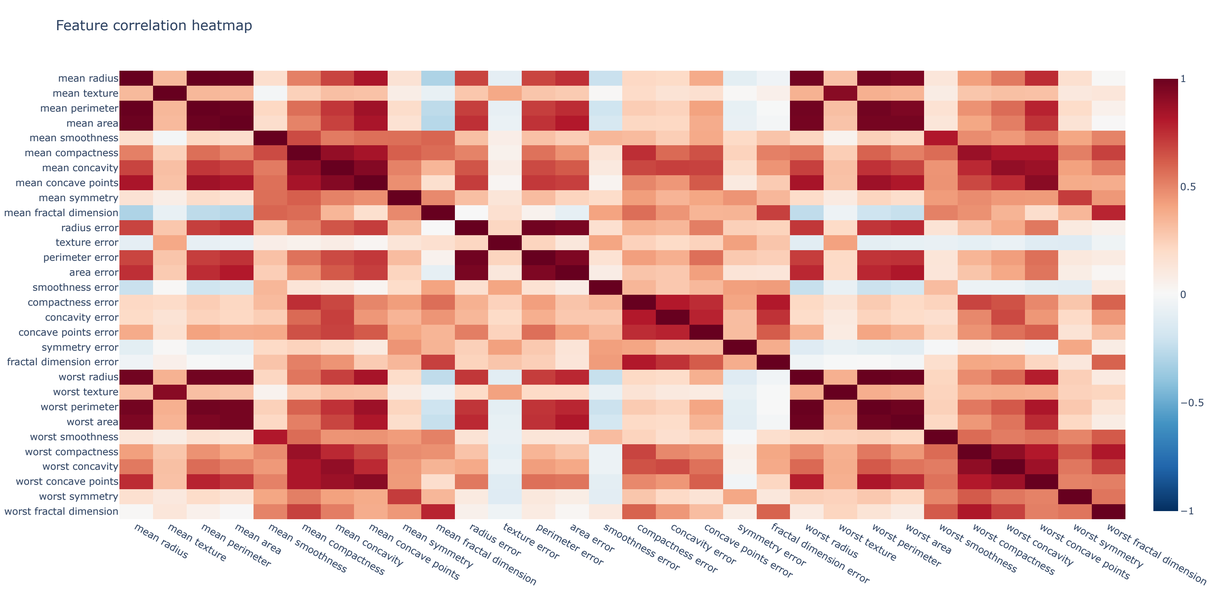
Dataset overview visualizations

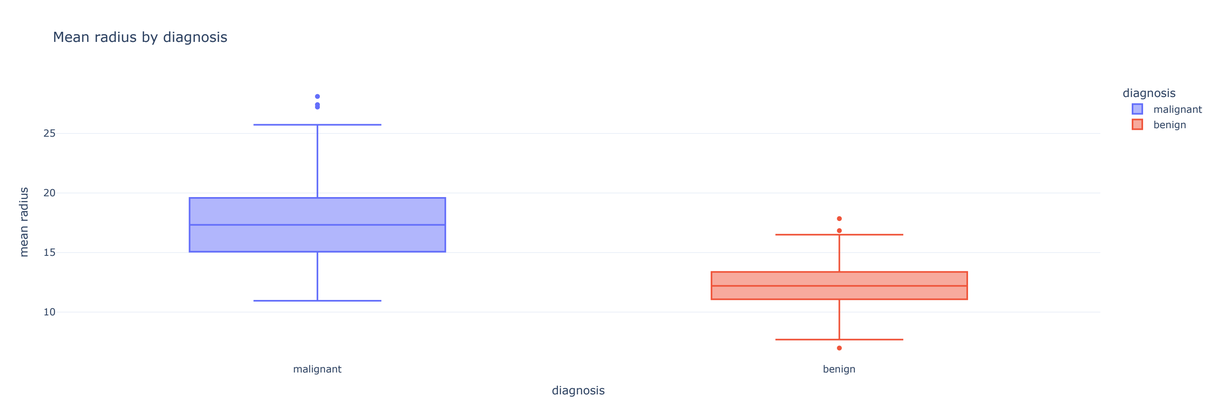
```
fig = px.histogram(df, x="diagnosis", color="diagnosis", title="Class distribution",
                   color_discrete_sequence=px.colors.qualitative.Set2)
fig.update_layout(template="plotly_white")
fig.show()

corr = X.corr(numeric_only=True)
fig = px.imshow(corr, color_continuous_scale="RdBu_r", zmin=-1, zmax=1,
                title="Feature correlation heatmap", aspect="auto")
fig.update_layout(height=750, template="plotly_white")
fig.show()

fig = px.box(df, x="diagnosis", y="mean radius", color="diagnosis",
             title="Mean radius by diagnosis", points="outliers")
fig.update_layout(template="plotly_white")
fig.show()
```







3. Preprocessing and train/test split

1. Split first, using stratification to preserve class proportions.
2. Fit the imputer and scaler **only on the training set** to avoid data leakage.
3. Apply the fitted transformations to the test set.
4. Standardize features because MLP optimization is sensitive to feature scale.

There are no categorical input features in this dataset, so categorical encoding is not applicable.

```
X_train_raw, X_test_raw, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=RANDOM_STATE, stratify=y
)

preprocessor = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])
X_train = preprocessor.fit_transform(X_train_raw).astype("float32")
X_test = preprocessor.transform(X_test_raw).astype("float32")
y_train_np = y_train.to_numpy().astype("float32")
y_test_np = y_test.to_numpy().astype("int32")

print("Training shape:", X_train.shape)
print("Testing shape:", X_test.shape)
print("Input neurons:", X_train.shape[1])
```

```
Training shape: (455, 30)
Testing shape: (114, 30)
Input neurons: 30
```

4. MLP architecture design

Baseline architecture:

- **Input:** 30 features.
- **Hidden layer 1:** 64 neurons, ReLU.
- **Hidden layer 2:** 32 neurons, ReLU.
- **Output:** 1 neuron, sigmoid.
- **Loss:** binary cross-entropy.
- **Optimizer:** Adam.

ReLU is a computationally simple non-linear activation that helps learn complex feature interactions. Sigmoid maps the output to a probability in `[0, 1]`, appropriate for binary classification. A threshold of 0.5 is used for class labels.

```
def build_mlp(input_dim, activation="relu", hidden=(64, 32), dropout=0.0):
    model = Sequential(name=f"MLP_{activation}_{'_'.join(map(str, hidden))}")
    model.add(Input(shape=(input_dim,), name="input"))
    for i, units in enumerate(hidden, start=1):
        model.add(Dense(units, activation=activation, name=f"hidden_{i}"))
        if dropout > 0:
            model.add(Dropout(dropout, name=f"dropout_{i}"))
    model.add(Dense(1, activation="sigmoid", name="output"))
    model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
                  loss="binary_crossentropy",
                  metrics=["accuracy"])
    return model

model = build_mlp(X_train.shape[1])
model.summary()
```

Model: "MLP_relu_64_32"

Layer (type)	Output Shape	Param #
hidden_1 (Dense)	(None, 64)	1,984
hidden_2 (Dense)	(None, 32)	2,080
output (Dense)	(None, 1)	33

Total params: 4,097 (16.00 KB)
Trainable params: 4,097 (16.00 KB)
Non-trainable params: 0 (0.00 B)

5. Train the model

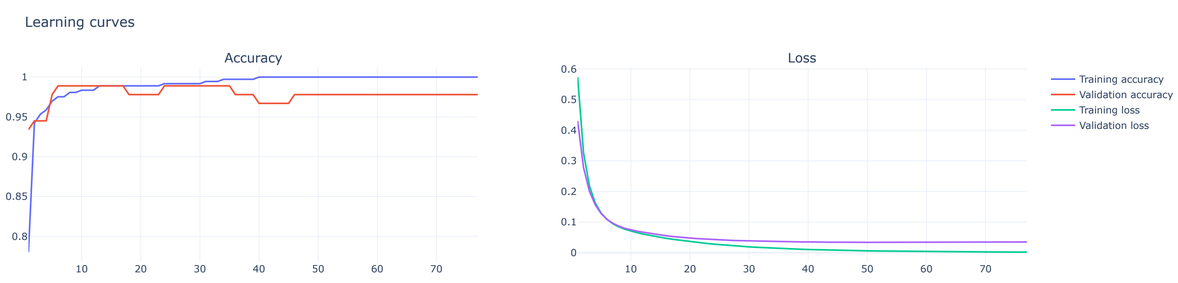
Early stopping monitors validation loss and restores the best weights. A validation split is drawn from the training data; the held-out test set is not used during training.

```
early_stop = EarlyStopping(monitor="val_loss", patience=20, restore_best_weights=True)
history = model.fit(
    X_train, y_train_np,
    validation_split=0.20,
    epochs=200,
    batch_size=32,
    callbacks=[early_stop],
    verbose=1
)
```

```
Epoch 49/200
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 1.0000 - loss: 0.0065 - val_accuracy: 0.9780 - val_loss: 0.0341
Epoch 50/200
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 1.0000 - loss: 0.0062 - val_accuracy: 0.9780 - val_loss: 0.0339
Epoch 51/200
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 1.0000 - loss: 0.0059 - val_accuracy: 0.9780 - val_loss: 0.0340
Epoch 52/200
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 1.0000 - loss: 0.0056 - val_accuracy: 0.9780 - val_loss: 0.0340
Epoch 53/200
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 1.0000 - loss: 0.0053 - val_accuracy: 0.9780 - val_loss: 0.0340
Epoch 54/200
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 1.0000 - loss: 0.0051 - val_accuracy: 0.9780 - val_loss: 0.0341
Epoch 55/200
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 1.0000 - loss: 0.0049 - val_accuracy: 0.9780 - val_loss: 0.0339
Epoch 56/200
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 1.0000 - loss: 0.0046 - val_accuracy: 0.9780 - val_loss: 0.0340
Epoch 57/200
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 1.0000 - loss: 0.0044 - val_accuracy: 0.9780 - val_loss: 0.0338
Epoch 58/200
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 1.0000 - loss: 0.0042 - val_accuracy: 0.9780 - val_loss: 0.0342
Epoch 59/200
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 1.0000 - loss: 0.0040 - val_accuracy: 0.9780 - val_loss: 0.0342
Epoch 60/200
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 1.0000 - loss: 0.0039 - val_accuracy: 0.9780 - val_loss: 0.0343
Epoch 61/200
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 1.0000 - loss: 0.0037 - val_accuracy: 0.9780 - val_loss: 0.0343
Epoch 62/200
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 1.0000 - loss: 0.0036 - val_accuracy: 0.9780 - val_loss: 0.0344
Epoch 63/200
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 1.0000 - loss: 0.0034 - val_accuracy: 0.9780 - val_loss: 0.0342
Epoch 64/200
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 1.0000 - loss: 0.0033 - val_accuracy: 0.9780 - val_loss: 0.0345
Epoch 65/200
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 1.0000 - loss: 0.0031 - val_accuracy: 0.9780 - val_loss: 0.0345
Epoch 66/200
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 1.0000 - loss: 0.0030 - val_accuracy: 0.9780 - val_loss: 0.0344
Epoch 67/200
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 1.0000 - loss: 0.0029 - val_accuracy: 0.9780 - val_loss: 0.0345
Epoch 68/200
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 1.0000 - loss: 0.0028 - val_accuracy: 0.9780 - val_loss: 0.0346
Epoch 69/200
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 1.0000 - loss: 0.0027 - val_accuracy: 0.9780 - val_loss: 0.0348
Epoch 70/200
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 1.0000 - loss: 0.0026 - val_accuracy: 0.9780 - val_loss: 0.0348
Epoch 71/200
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 1.0000 - loss: 0.0025 - val_accuracy: 0.9780 - val_loss: 0.0350
Epoch 72/200
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 1.0000 - loss: 0.0024 - val_accuracy: 0.9780 - val_loss: 0.0347
Epoch 73/200
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 1.0000 - loss: 0.0023 - val_accuracy: 0.9780 - val_loss: 0.0349
Epoch 74/200
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 1.0000 - loss: 0.0022 - val_accuracy: 0.9780 - val_loss: 0.0352
Epoch 75/200
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 1.0000 - loss: 0.0021 - val_accuracy: 0.9780 - val_loss: 0.0352
Epoch 76/200
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 1.0000 - loss: 0.0021 - val_accuracy: 0.9780 - val_loss: 0.0349
Epoch 77/200
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 1.0000 - loss: 0.0020 - val_accuracy: 0.9780 - val_loss: 0.0351
```

Training curves

```
hist = pd.DataFrame(history.history)
hist["epoch"] = np.arange(1, len(hist) + 1)
fig = make_subplots(rows=1, cols=2, subplot_titles=("Accuracy", "Loss"))
fig.add_trace(go.Scatter(x=hist.epoch, y=hist.accuracy, name="Training accuracy"), row=1, col=1)
fig.add_trace(go.Scatter(x=hist.epoch, y=hist.val_accuracy, name="Validation accuracy"), row=1, col=1)
fig.add_trace(go.Scatter(x=hist.epoch, y=hist.loss, name="Training loss"), row=1, col=2)
fig.add_trace(go.Scatter(x=hist.epoch, y=hist.val_loss, name="Validation loss"), row=1, col=2)
fig.update_layout(template="plotly_white", height=420, title="Learning curves")
fig.show()
```



6. Model summary and trainable parameter mathematics

For a Dense layer:

[\text{Parameters} = (\text{input connections} \times \text{neurons}) + \text{biases}]

Equivalently, for `n_in` inputs and `n_out` neurons:

[P = (n_{in} \times n_{out}) + n_{out}]

The input layer has no trainable parameters. Dropout layers (if present) also have no trainable parameters. The output Dense layer has one sigmoid neuron.

```
def dense_parameter_table(keras_model):
    rows = []
    previous_units = None
    for layer in keras_model.layers:
        if isinstance(layer, Dense):
            units = layer.units
            input_units = int(layer.kernel.shape[0])
            weights = input_units * units
            biases = units
            rows.append({
                "Layer": layer.name,
                "Input connections": input_units,
                "Neurons": units,
                "Weights": weights,
                "Biases": biases,
                "Formula": f"({input_units} × {units}) + {units}",
                "Trainable parameters": weights + biases
            })
    return pd.DataFrame(rows)

param_table = dense_parameter_table(model)
display(param_table)
print("Calculated total:", int(param_table["Trainable parameters"].sum()))
print("Keras total params:", model.count_params())
print("Trainable params:", sum(int(np.prod(v.shape)) for v in model.trainable_weights))
print("Non-trainable params:", sum(int(np.prod(v.shape)) for v in model.non_trainable_weights))
```

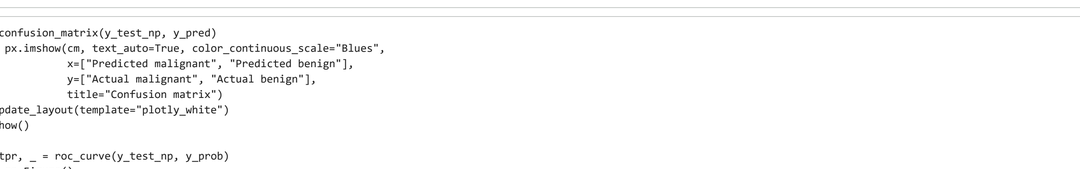
	Layer	Input connections	Neurons	Weights	Biases	Formula	Trainable parameters
0	hidden_1	30	64	1920	64	(30 × 64) + 64	1984
1	hidden_2	64	32	2048	32	(64 × 32) + 32	2080
2	output	32	1	32	1	(32 × 1) + 1	33

```
Calculated total: 4097
Keras total params: 4097
Trainable params: 4097
Non-trainable params: 0
```

Next steps: Generate code with `param_table` | New interactive sheet

Architecture diagram (interactive Plotly)

```
layer_names = ["Input (30)", "Dense (64)", "Dense (32)", "Output (1)"]
fig = go.Figure()
for i, name in enumerate(layer_names):
    fig.add_trace(go.Scatter(x=[i], y=[0], mode="markers+text",
                             marker=dict(size=48, color=px.colors.qualitative.Bold[i]),
                             text=[name], textposition="bottom center",
                             showlegend=False))
for i in range(len(layer_names)-1):
    fig.add_annotation(x=i+1, y=0, ax=i, ay=0, xref="x", yref="y", axref="x", ayref="y",
                       text="", showarrow=True, arrowhead=3, arrowsize=1.5)
fig.update_layout(title="MLP architecture", xaxis=dict(visible=False), yaxis=dict(visible=False),
                  template="plotly_white", height=300)
fig.show()
```



7. Model evaluation

Report accuracy, precision, recall, F1-score, confusion matrix, and ROC-AUC. The positive class is benign (`1`) as encoded by the source dataset. For a real medical application, class definitions and error costs must be reviewed with domain experts.

```
test_loss, test_accuracy = model.evaluate(X_test, y_test_np, verbose=0)
y_prob = model.predict(X_test, verbose=0).ravel()
y_pred = (y_prob >= 0.5).astype(int)

metrics = {
    "Accuracy": accuracy_score(y_test_np, y_pred),
    "Precision": precision_score(y_test_np, y_pred, zero_division=0),
    "Recall": recall_score(y_test_np, y_pred, zero_division=0),
    "F1-score": f1_score(y_test_np, y_pred, zero_division=0),
    "ROC-AUC": roc_auc_score(y_test_np, y_prob),
    "Test loss": test_loss
}
display(pd.DataFrame([metrics]).T.rename(columns={0: "Score"}).style.format("{:.4f}"))
print(classification_report(y_test_np, y_pred, target_names=["malignant", "benign"]))
```

	Score
Accuracy	0.9561
Precision	0.9855
Recall	0.9444
F1-score	0.9645
ROC-AUC	0.9917
Test loss	0.1289

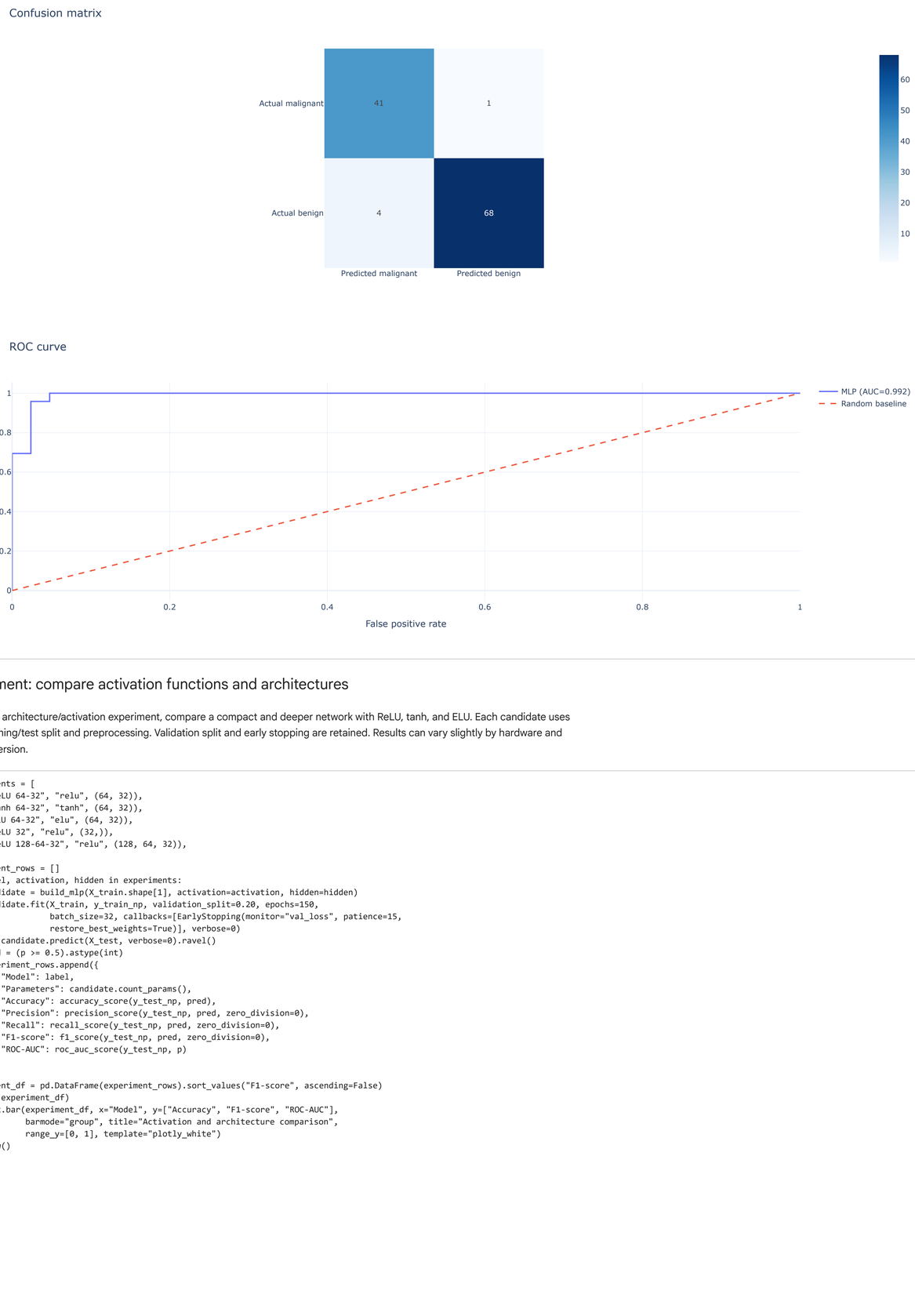
```
              precision    recall  f1-score   support

   malignant       0.91      0.98      0.94        42
      benign       0.99      0.94      0.96        72

    accuracy                           0.96       114
   macro avg       0.95      0.96      0.95       114
weighted avg       0.96      0.96      0.96       114
```

```
cm = confusion_matrix(y_test_np, y_pred)
fig = px.imshow(cm, text_auto=True, color_continuous_scale="Blues",
                x=["Predicted malignant", "Predicted benign"],
                y=["Actual malignant", "Actual benign"],
                title="Confusion matrix")
fig.update_layout(template="plotly_white")
fig.show()

fpr, tpr, _ = roc_curve(y_test_np, y_prob)
fig = go.Figure()
fig.add_trace(go.Scatter(x=fpr, y=tpr, mode="lines", name=f"MLP (AUC={metrics['ROC-AUC']:.3f})"))
fig.add_trace(go.Scatter(x=[0,1], y=[0,1], mode="lines", name="Random baseline",
                         line=dict(dash="dash")))
fig.update_layout(title="ROC curve", xaxis_title="False positive rate",
                  yaxis_title="True positive rate", template="plotly_white")
fig.show()
```



8. Experiment: compare activation functions and architectures

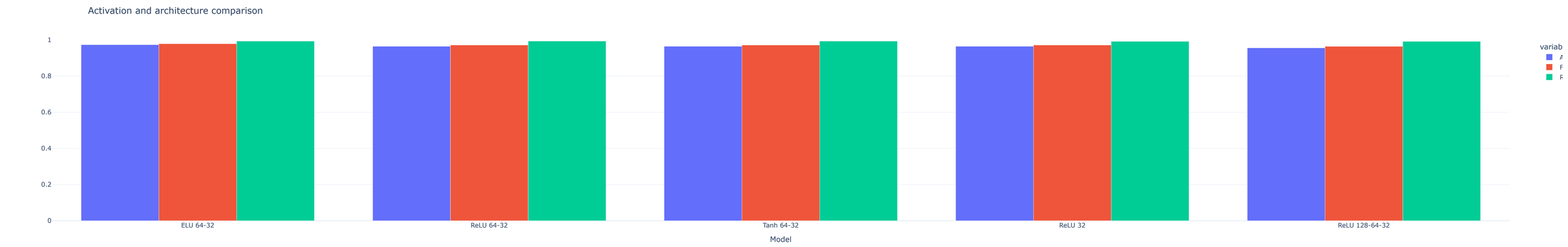
To satisfy the architecture/activation experiment, compare a compact and deeper network with ReLU, tanh, and ELU. Each candidate uses the same training/test split and preprocessing. Validation split and early stopping are retained. Results can vary slightly by hardware and TensorFlow version.

```
experiments = [
    ("ReLU 64-32", "relu", (64, 32)),
    ("Tanh 64-32", "tanh", (64, 32)),
    ("ELU 64-32", "elu", (64, 32)),
    ("ReLU 32", "relu", (32,)),
    ("ReLU 128-64-32", "relu", (128, 64, 32)),
]
experiment_rows = []
for label, activation, hidden in experiments:
    candidate = build_mlp(X_train.shape[1], activation=activation, hidden=hidden)
    candidate.fit(X_train, y_train_np, validation_split=0.20, epochs=150,
                  batch_size=32, callbacks=[EarlyStopping(monitor="val_loss", patience=15,
                  restore_best_weights=True)], verbose=0)
    p = candidate.predict(X_test, verbose=0).ravel()
    pred = (p >= 0.5).astype(int)
    experiment_rows.append({
        "Model": label,
        "Parameters": candidate.count_params(),
        "Accuracy": accuracy_score(y_test_np, pred),
        "Precision": precision_score(y_test_np, pred, zero_division=0),
        "Recall": recall_score(y_test_np, pred, zero_division=0),
        "F1-score": f1_score(y_test_np, pred, zero_division=0),
        "ROC-AUC": roc_auc_score(y_test_np, p)
    })

experiment_df = pd.DataFrame(experiment_rows).sort_values("F1-score", ascending=False)
display(experiment_df)
fig = px.bar(experiment_df, x="Model", y=["Accuracy", "F1-score", "ROC-AUC"],
             barmode="group", title="Activation and architecture comparison",
             range_y=[0, 1], template="plotly_white")
fig.show()
```

```
WARNING:tensorflow:5 out of the last 9 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x7c14216442c0> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead
WARNING:tensorflow:6 out of the last 12 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x7c14216442c0> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instea
```

	Model	Parameters	Accuracy	Precision	Recall	F1-score	ROC-AUC
2	ELU 64-32	4097	0.973684	0.985915	0.972222	0.979021	0.993386
0	ReLU 64-32	4097	0.964912	0.985714	0.958333	0.971831	0.993717
1	Tanh 64-32	4097	0.964912	0.985714	0.958333	0.971831	0.993386
3	ReLU 32	1025	0.964912	0.985714	0.958333	0.971831	0.992063
4	ReLU 128-64-32	14337	0.956140	0.985507	0.944444	0.964539	0.991733



Next steps: Generate code with `experiment_df` New interactive sheet

9. Interpretation: overfitting and underfitting

- **Overfitting indicators:** training accuracy substantially higher than validation accuracy, or training loss continuing to decrease while validation loss rises.
- **Underfitting indicators:** both training and validation accuracy remain low, with both losses high.
- **Reasonable generalization:** training and validation curves remain close and validation loss does not diverge strongly.
- The test metrics estimate performance on held-out data. They do not establish clinical safety or suitability.

10. Self-learning questions and answers

Q1. What is an MLP?
An MLP is a feed-forward artificial neural network containing an input layer, one or more fully connected hidden layers, and an output layer.

Q2. Why is feature scaling important for an MLP?
Features on very different scales can lead to poorly conditioned optimization and slower or unstable learning. Standardization places features on a comparable scale.

Q3. Why is ReLU used in hidden layers?
ReLU introduces non-linearity and is computationally inexpensive. It often supports efficient gradient-based training.

Q4. Why is sigmoid used in the output layer?
For binary classification, sigmoid converts the output logit to a value between 0 and 1 that can be interpreted as the estimated probability of the positive class.

Q5. Why use binary cross-entropy?
It is a standard loss for binary classification and penalizes disagreement between predicted probabilities and binary labels.

Q6. What is the purpose of train/test splitting?
The training set is used to fit the model; the held-out test set estimates performance on unseen observations.

Q7. What is data leakage?
Data leakage occurs when information from the test set influences training or preprocessing. Fitting the scaler on all observations is one example; fit preprocessing on training data only.

Q8. How are Dense-layer parameters calculated?
For `n_in` inputs and `n_out` neurons, parameters equal `n_in × n_out` weights plus `n_out` biases.

Q9. What is the difference between trainable and non-trainable parameters?
Trainable parameters are updated by the optimizer. Non-trainable parameters are stored in the model but are not updated during training. This model has no non-trainable parameters.

Q10. What does a confusion matrix show?
It counts correct and incorrect predictions by actual and predicted class, revealing the types of classification errors.

Q11. What is the difference between precision and recall?
Precision is the fraction of predicted positives that are truly positive. Recall is the fraction of actual positives that are detected.

Q12. How can overfitting be reduced?
Use early stopping, dropout, regularization, a simpler architecture, more representative training data, or data augmentation where appropriate.